

UrbanSense [0.4cm] *Análisis de Contaminación Acústica* Urbana [0.2cm] Universidad

Jorge Emanuel Frias Martin Del Campo \ Matrícula: 23310025 \ Grupo: 7B

17 de August de 2026

Contents

1	Introducción	3
2	Generación del Dataset Sintético (01_generar_datos.R)	3
2.1	Descripción	3
2.2	Características base por zona	5
2.3	Archivos auxiliares generados	5
3	Importación y Limpieza de Datos (02_importar_limpiar.R)	6
3.1	Importación desde múltiples formatos	6
3.2	Proceso de Limpieza	7
3.2.1	Tipos de datos y duplicados	7
3.2.2	Valores faltantes (NA)	8
3.2.3	Corrección de categorías y outliers	8
3.2.4	Demostración de Joins	9
4	Análisis Exploratorio de Datos (03_analisis.R)	10
4.1	Operaciones dplyr	10
4.2	Estadísticas descriptivas de decibeles	11
4.3	Visualizaciones	11
4.3.1	Distribución general de ruido	11
4.3.2	Distribución por tipo de zona	13
4.3.3	Ruido promedio por zona	15
4.3.4	Relación tráfico — ruido	16
4.3.5	Heatmap día-hora	17
5	Análisis de Componentes Principales — PCA (04_pca.R)	18
5.1	Loadings	19
5.2	Scree plot — varianza explicada	19
5.3	Gráfica PC1 vs PC2	20

6	Series de Tiempo y Modelo ARIMA (05_series_tiempo.R)	21
6.1	Construcción de la serie diaria	21
6.2	Gráfica temporal con tendencia	21
6.3	Autocorrelación (ACF)	22
6.4	Descomposición STL	22
6.5	Modelo ARIMA y Predicción	23
7	Análisis de Texto — Datos No Estructurados (06_no_estructurados.R)	25
7.1	Tokenización	25
7.2	Frecuencia de palabras	25
7.3	Gráfica de palabras frecuentes	26
8	Conclusiones	27

1 Introducción

UrbanSense es un proyecto de análisis de datos orientado a comprender los patrones de contaminación acústica en una ciudad. A partir de ~1,500 mediciones sintéticas pero realistas de nivel de ruido, tráfico, temperatura y humedad en distintas zonas urbanas de Guadalajara, se aplica un pipeline completo de ciencia de datos:

1. **Generación de datos** con errores intencionales para practicar limpieza.
 2. **Importación y limpieza** desde múltiples formatos (CSV, Excel, JSON, XML, SQLite).
 3. **Análisis exploratorio** con estadísticas descriptivas y visualizaciones.
 4. **Reducción de dimensionalidad** mediante Análisis de Componentes Principales (PCA).
 5. **Modelado de series de tiempo** con ARIMA.
 6. **Análisis de texto** sobre comentarios de campo (datos no estructurados).
-

2 Generación del Dataset Sintético (01_generar_datos.R)

2.1 Descripción

Se generaron $n = 1,500$ mediciones con relaciones realistas entre variables. Cada medición incluye: zona, tipo de zona, coordenadas geográficas, decibeles, índice de tráfico, temperatura, humedad y un comentario de texto libre.

```
library(tidyverse)
library(writexl)
library(jsonlite)
library(DBI)
library(RSQLite)
library(lubridate)

set.seed(123)
n <- 1500

zonas_info <- tibble(
  zona      = c("Centro", "Americana", "Providencia", "Chapalita",
               "Zona Industrial", "Tlaquepaque Centro"),
  tipo_zona = c("Comercial", "Mixta", "Residencial", "Residencial",
               "Industrial", "Comercial"),
  ruido_base = c(72, 65, 55, 50, 82, 70),
  traf_base  = c(72, 62, 40, 32, 55, 68),
  lat        = c(20.6737, 20.6666, 20.6612, 20.6580, 20.6450, 20.6420),
  lng        = c(-103.3440, -103.3810, -103.3890, -103.4020, -103.3200, -103.3120)
)

fechas      <- seq(as.Date("2025-02-01"), as.Date("2025-07-31"), by = "day")
fecha_vec   <- sample(fechas, n, replace = TRUE)
hora_vec    <- sample(0:23, n, replace = TRUE)
zona_idx    <- sample(1:6, n, replace = TRUE)

zona_vec    <- zonas_info$zona[zona_idx]
```

```

tipo_vec   <- zonas_info$tipo_zona[zona_idx]
lat_vec    <- zonas_info$lat[zona_idx]
lng_vec    <- zonas_info$lng[zona_idx]
ruido_base <- zonas_info$ruido_base[zona_idx]
traf_base  <- zonas_info$traf_base[zona_idx]

factor_hora <- case_when(
  hora_vec %in% 7:9 ~ 1.30,
  hora_vec %in% 17:19 ~ 1.25,
  hora_vec %in% 0:5 ~ 0.50,
  TRUE ~ 1.00
)

es_finde    <- wday(fecha_vec) %in% c(1, 7)
factor_finde <- ifelse(es_finde, 0.85, 1.0)

trafico_vec <- as.integer(round(pmax(0, pmin(100,
  traf_base * factor_hora * factor_finde + rnorm(n, 0, 8)
))))

decibeles_vec <- round(pmax(30, pmin(115,
  ruido_base * factor_hora * factor_finde + 0.12 * trafico_vec + rnorm(n, 0, 4)
)), 1)

temperatura_vec <- round(
  22 + ifelse(hora_vec %in% 12:16, 8,
    ifelse(hora_vec %in% 0:5, -4, 2)) + rnorm(n, 0, 2), 1)

humedad_vec <- round(pmax(20, pmin(95, 60 + rnorm(n, 0, 12))), 1)

comentarios_pool <- c(
  "Mucho tráfico", "Zona tranquila", "Muchos camiones",
  "Ruido por construcción", "Música alta", "Tráfico intenso",
  "Sin incidencias", "Obras en la zona", "Mercado cercano",
  "Mucho ruido", "Zona ruidosa", "Poco tráfico"
)

comentario_vec <- sample(comentarios_pool, n, replace = TRUE)

mediciones <- tibble(
  id      = 1:n,
  fecha   = fecha_vec,
  hora    = hora_vec,
  zona    = zona_vec,
  tipo_zona = tipo_vec,
  lat     = lat_vec,
  lng     = lng_vec,
  decibeles = decibeles_vec,
  trafico  = trafico_vec,
  temperatura = temperatura_vec,
  humedad  = humedad_vec,
  comentario = comentario_vec
)

```

```

# Errores intencionales para demostrar limpieza
idx_inc <- sample(1:n, 30)
mediciones$zona[idx_inc[1:10]] <- "centro"
mediciones$zona[idx_inc[11:20]] <- "CENTRO"
mediciones$zona[idx_inc[21:30]] <- "zona industrial"

idx_imp <- sample(1:n, 3)
mediciones$decibeles[idx_imp] <- 350

idx_out <- sample(setdiff(1:n, idx_imp), 5)
mediciones$decibeles[idx_out] <- c(115, 118, 120, 119, 116)

for (col in c("decibeles", "trafico", "temperatura", "humedad")) {
  mediciones[[col]][sample(1:n, 4)] <- NA
}
mediciones$zona[sample(1:n, 3)] <- NA
mediciones$tipo_zona[sample(1:n, 3)] <- NA

idx_dup <- sample(1:n, 10)
mediciones <- bind_rows(mediciones, mediciones[idx_dup, ])

dir.create("data/raw", recursive = TRUE, showWarnings = FALSE)
dir.create("data/processed", recursive = TRUE, showWarnings = FALSE)

write_csv(mediciones, "data/raw/mediciones.csv")

```

2.2 Características base por zona

Table 1: Parámetros base por zona geográfica

Zona	Tipo	Ruido base (dB)	Tráfico base
Centro	Comercial	72	72
Americana	Mixta	65	62
Providencia	Residencial	55	40
Chapalita	Residencial	50	32
Zona Industrial	Industrial	82	55
Tlaquepaque Centro	Comercial	70	68

2.3 Archivos auxiliares generados

Se exportaron también los siguientes formatos para demostrar importación múltiple:

```

zonas_cat <- tibble(
  zona = c("Centro", "Americana", "Providencia",
           "Chapalita", "Zona Industrial", "Tlaquepaque Centro"),
  municipio = rep("Guadalajara", 5) %>% c("Tlaquepaque"),
  tipo_zona = c("Comercial", "Mixta", "Residencial",
               "Residencial", "Industrial", "Comercial")
)
writexl::write_xlsx(zonas_cat, "data/raw/zonas.xlsx")

```

```

sensores_list <- list(
  list(sensor_id="S001", nombre="Sensor Centro Norte", zona="Centro", estado="activo"),
  list(sensor_id="S002", nombre="Sensor Americana", zona="Americana", estado="activo"),
  list(sensor_id="S003", nombre="Sensor Providencia", zona="Providencia", estado="inactivo"),
  list(sensor_id="S004", nombre="Sensor Industrial A", zona="Zona Industrial", estado="activo"),
  list(sensor_id="S005", nombre="Sensor Tlaquepaque", zona="Tlaquepaque Centro", estado="activo"),
  list(sensor_id="S006", nombre="Sensor Chapalita", zona="Chapalita", estado="mantenimiento")
)
jsonlite::write_json(sensores_list, "data/raw/sensores.json", pretty = TRUE)

xml_lines <- c('<?xml version="1.0" encoding="UTF-8"?>', "<zonas>")
for (i in seq_len(nrow(zonas_cat))) {
  xml_lines <- c(xml_lines, " <zona>",
    paste0(" <nombre>", zonas_cat$zona[i], "</nombre>"),
    paste0(" <municipio>", zonas_cat$municipio[i], "</municipio>"),
    paste0(" <tipo>", zonas_cat$tipo_zona[i], "</tipo>"),
    " </zona>")
}
xml_lines <- c(xml_lines, "</zonas>")
writeLines(xml_lines, "data/raw/zonas.xml", useBytes = FALSE)

con <- DBI::dbConnect(RSQLite::SQLite(), "data/raw/urbansense.sqlite")
sensores_df <- data.frame(
  sensor_id = c("S001", "S002", "S003", "S004", "S005", "S006"),
  nombre = c("Sensor Centro Norte", "Sensor Americana", "Sensor Providencia",
    "Sensor Industrial A", "Sensor Tlaquepaque", "Sensor Chapalita"),
  zona = c("Centro", "Americana", "Providencia",
    "Zona Industrial", "Tlaquepaque Centro", "Chapalita"),
  estado = c("activo", "activo", "inactivo", "activo", "activo", "mantenimiento"),
  stringsAsFactors = FALSE
)
DBI::dbWriteTable(con, "sensores", sensores_df, overwrite = TRUE)
DBI::dbDisconnect(con)

cat("Archivos generados: mediciones.csv, zonas.xlsx, sensores.json, zonas.xml, urbansense.sqlite\n")

## Archivos generados: mediciones.csv, zonas.xlsx, sensores.json, zonas.xml, urbansense.sqlite

cat("Dataset bruto:", nrow(mediciones), "filas x", ncol(mediciones), "columnas\n")

## Dataset bruto: 1510 filas x 12 columnas

```

3 Importación y Limpieza de Datos (02_importar_limpiar.R)

3.1 Importación desde múltiples formatos

```

library(readxl)
library(xml2)

mediciones_raw <- read_csv("data/raw/mediciones.csv", show_col_types = FALSE)
cat("CSV cargado:", nrow(mediciones_raw), "filas,", ncol(mediciones_raw), "columnas\n")

## CSV cargado: 1510 filas, 12 columnas

zonas_excel <- readxl::read_excel("data/raw/zonas.xlsx")
cat("Excel cargado:", nrow(zonas_excel), "filas\n")

## Excel cargado: 6 filas

sensores_json <- jsonlite::fromJSON("data/raw/sensores.json")
cat("JSON cargado:", nrow(sensores_json), "sensores\n")

## JSON cargado: 6 sensores

xml_doc <- xml2::read_xml("data/raw/zonas.xml")
nodos_zona <- xml_find_all(xml_doc, "//zona")
zonas_xml <- tibble(
  nombre = sapply(nodos_zona, function(n) xml_text(xml_find_first(n, "nombre"))),
  municipio = sapply(nodos_zona, function(n) xml_text(xml_find_first(n, "municipio"))),
  tipo = sapply(nodos_zona, function(n) xml_text(xml_find_first(n, "tipo")))
)
cat("XML cargado:", nrow(zonas_xml), "zonas\n")

## XML cargado: 6 zonas

con <- DBI::dbConnect(RSQLite::SQLite(), "data/raw/urbansense.sqlite")
sensores_sql <- DBI::dbGetQuery(con, "SELECT * FROM sensores WHERE estado = 'activo'")
DBI::dbDisconnect(con)
cat("SQLite - sensores activos:", nrow(sensores_sql), "\n")

## SQLite - sensores activos: 4

```

3.2 Proceso de Limpieza

3.2.1 Tipos de datos y duplicados

```

mediciones <- mediciones_raw %>%
  mutate(
    fecha = as.Date(fecha),
    hora = as.integer(hora),
    decibeles = as.numeric(decibeles),
    trafico = as.numeric(trafico),
    zona = as.character(zona),

```

```

    tipo_zona = as.character(tipo_zona)
  )

n_antes  <- nrow(mediciones)
mediciones <- distinct(mediciones)
cat("Filas duplicadas eliminadas:", n_antes - nrow(mediciones), "\n")

```

```
## Filas duplicadas eliminadas: 10
```

```
cat("Filas tras distinct():", nrow(mediciones), "\n")
```

```
## Filas tras distinct(): 1500
```

3.2.2 Valores faltantes (NA)

```
cat("NA por columna antes de imputar:\n")
```

```
## NA por columna antes de imputar:
```

```
na_tabla <- colSums(is.na(mediciones))
print(na_tabla[na_tabla > 0])
```

```
##      zona  tipo_zona  decibeles  trafico temperatura  humedad
##      3          3          4          4          4          4
```

```

moda <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) == 0) return(NA)
  unique(x)[which.max(tabulate(match(x, unique(x))))]
}

for (col in c("decibeles", "trafico", "temperatura", "humedad")) {
  val_mediana <- median(mediciones[[col]], na.rm = TRUE)
  mediciones[[col]][is.na(mediciones[[col]])] <- val_mediana
}

mediciones$zona[is.na(mediciones$zona)] <- moda(mediciones$zona)
mediciones$tipo_zona[is.na(mediciones$tipo_zona)] <- moda(mediciones$tipo_zona)

cat("\nNA restantes:", sum(is.na(mediciones)), "\n")

```

```
##
## NA restantes: 0
```

3.2.3 Corrección de categorías y outliers


```

mediciones <- mediciones %>%
  mutate(zona = str_to_title(str_trim(zona)))

cat("Categorías únicas en zona después de normalizar:\n")

## Categorías únicas en zona después de normalizar:

print(sort(unique(mediciones$zona)))

## [1] "Americana"          "Centro"              "Chapalita"
## [4] "Providencia"        "Tlaquepaque Centro" "Zona Industrial"

n_imposibles <- sum(mediciones$decibeles > 194, na.rm = TRUE)
cat("\nValores físicamente imposibles (>194 dB):", n_imposibles, "\n")

##
## Valores físicamente imposibles (>194 dB): 3

# Eliminar los valores imposibles
mediciones <- mediciones %>%
  filter(is.na(decibeles) | decibeles <= 194)

cat("Filas finales tras eliminar imposibles:", nrow(mediciones), "\n")

## Filas finales tras eliminar imposibles: 1497

write_csv(mediciones, "data/processed/mediciones_limpias.csv")
cat("Dataset limpio guardado en data/processed/mediciones_limpias.csv\n")

## Dataset limpio guardado en data/processed/mediciones_limpias.csv

```

3.2.4 Demostración de Joins

```

mediciones_join <- mediciones %>%
  mutate(zona_norm = str_to_title(str_trim(zona))) %>%
  inner_join(zonas_excel, by = c("zona_norm" = "zona"))
cat("Filas tras inner_join con catálogo de zonas:", nrow(mediciones_join), "\n")

## Filas tras inner_join con catálogo de zonas: 1497

ejemplo_a <- tibble(id = 1:3, valor = c(10, 20, 30))
ejemplo_b <- tibble(id = 2:4, etiqueta = c("B", "C", "D"))
demo_fulljoin <- full_join(ejemplo_a, ejemplo_b, by = "id")
cat("\nEjemplo full_join (NA donde no hay coincidencia):\n")

##
## Ejemplo full_join (NA donde no hay coincidencia):

```

```
print(demo_fulljoin)
```

```
## # A tibble: 4 x 3
##       id valor etiqueta
##   <int> <dbl> <chr>
## 1     1     10 <NA>
## 2     2     20 B
## 3     3     30 C
## 4     4      NA D
```

4 Análisis Exploratorio de Datos (03_analisis.R)

4.1 Operaciones dplyr

```
mediciones <- read_csv("data/processed/mediciones_limpias.csv", show_col_types = FALSE) %>%
  mutate(fecha = as.Date(fecha), tipo_zona = as.factor(tipo_zona))
```

```
alta_contaminacion <- mediciones %>% filter(decibeles > 80)
cat("Mediciones > 80 dB:", nrow(alta_contaminacion), "\n")
```

```
## Mediciones > 80 dB: 398
```

```
ruido_por_zona <- mediciones %>%
  group_by(zona) %>%
  summarise(
    decibeles_prom = round(mean(decibeles, na.rm = TRUE), 2),
    trafico_prom   = round(mean(trafico,   na.rm = TRUE), 2),
    n_mediciones   = n(),
    .groups        = "drop"
  ) %>%
  arrange(desc(decibeles_prom))

knitr::kable(ruido_por_zona,
  col.names = c("Zona", "dB Promedio", "Tráfico Promedio", "N"),
  booktabs = TRUE,
  caption   = "Ruido y tráfico promedio por zona")
```

Table 2: Ruido y tráfico promedio por zona

Zona	dB Promedio	Tráfico Promedio	N
Zona Industrial	79.50	49.61	231
Centro	72.81	63.72	262
Tlaquepaque Centro	72.30	63.53	231
Americana	64.72	55.10	293
Providencia	56.77	36.88	224
Chapalita	51.37	29.59	256

4.2 Estadísticas descriptivas de decibeles

```
stats_db <- mediciones %>%
  summarise(
    Media    = round(mean(decibeles,      na.rm=TRUE), 2),
    Mediana  = round(median(decibeles,    na.rm=TRUE), 2),
    DE       = round(sd(decibeles,        na.rm=TRUE), 2),
    Mínimo   = round(min(decibeles,       na.rm=TRUE), 2),
    Máximo   = round(max(decibeles,       na.rm=TRUE), 2),
    Q1       = round(quantile(decibeles, 0.25, na.rm=TRUE), 2),
    Q3       = round(quantile(decibeles, 0.75, na.rm=TRUE), 2),
    IQR      = round(IQR(decibeles,       na.rm=TRUE), 2)
  )

knitr::kable(t(stats_db), col.names = "Valor",
  booktabs = TRUE, caption = "Estadísticas descriptivas - Decibeles")
```

Table 3: Estadísticas descriptivas — Decibeles

	Valor
Media	66.11
Mediana	68.35
DE	22.71
Mínimo	30.00
Máximo	120.00
Q1	46.90
Q3	81.00
IQR	34.10

4.3 Visualizaciones

4.3.1 Distribución general de ruido

```
ggplot(mediciones, aes(x = decibeles)) +
  geom_histogram(binwidth = 3, fill = "#2196F3", color = "white", alpha = 0.85) +
  labs(title = "Distribución de niveles de ruido",
    x = "Decibeles (dB)", y = "Frecuencia") +
  theme_minimal()
```

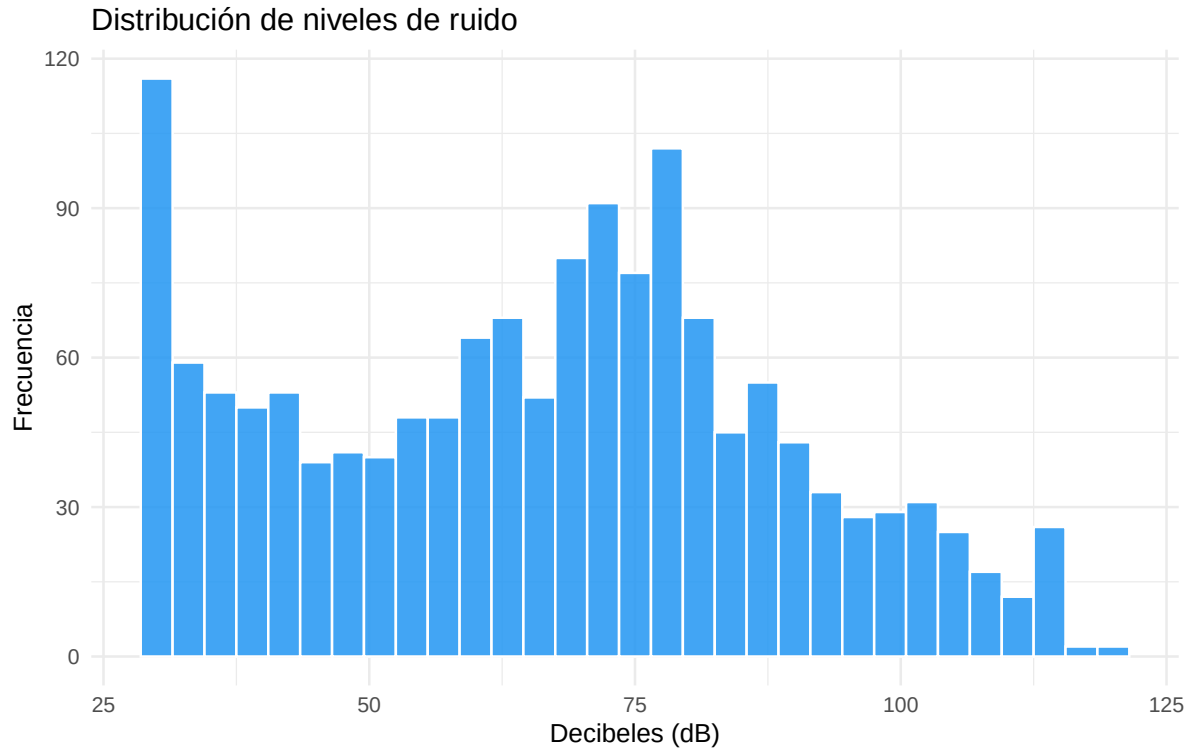


Figure 1: Distribución de niveles de ruido registrados

```
ggplot(mediciones, aes(x = decibeles)) +  
  geom_density(fill = "#4CAF50", color = "#2E7D32", alpha = 0.65) +  
  labs(title = "Densidad de niveles de ruido",  
        x = "Decibeles (dB)", y = "Densidad") +  
  theme_minimal()
```

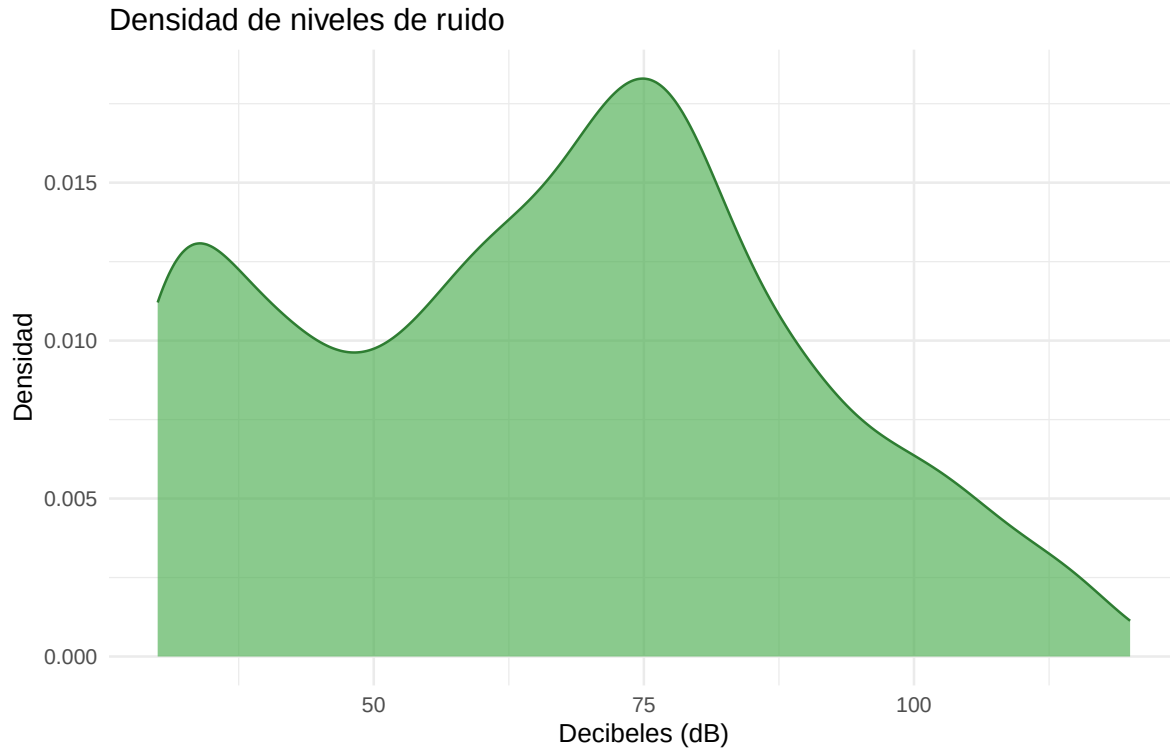


Figure 2: Curva de densidad del nivel de ruido

4.3.2 Distribución por tipo de zona

```
ggplot(mediciones, aes(x = tipo_zona, y = decibeles, fill = tipo_zona)) +  
  geom_boxplot(alpha = 0.8, outlier.color = "red", outlier.size = 1) +  
  labs(title = "Distribución de ruido por tipo de zona",  
        x = "Tipo de zona", y = "Decibeles (dB)") +  
  theme_minimal() + theme(legend.position = "none")
```

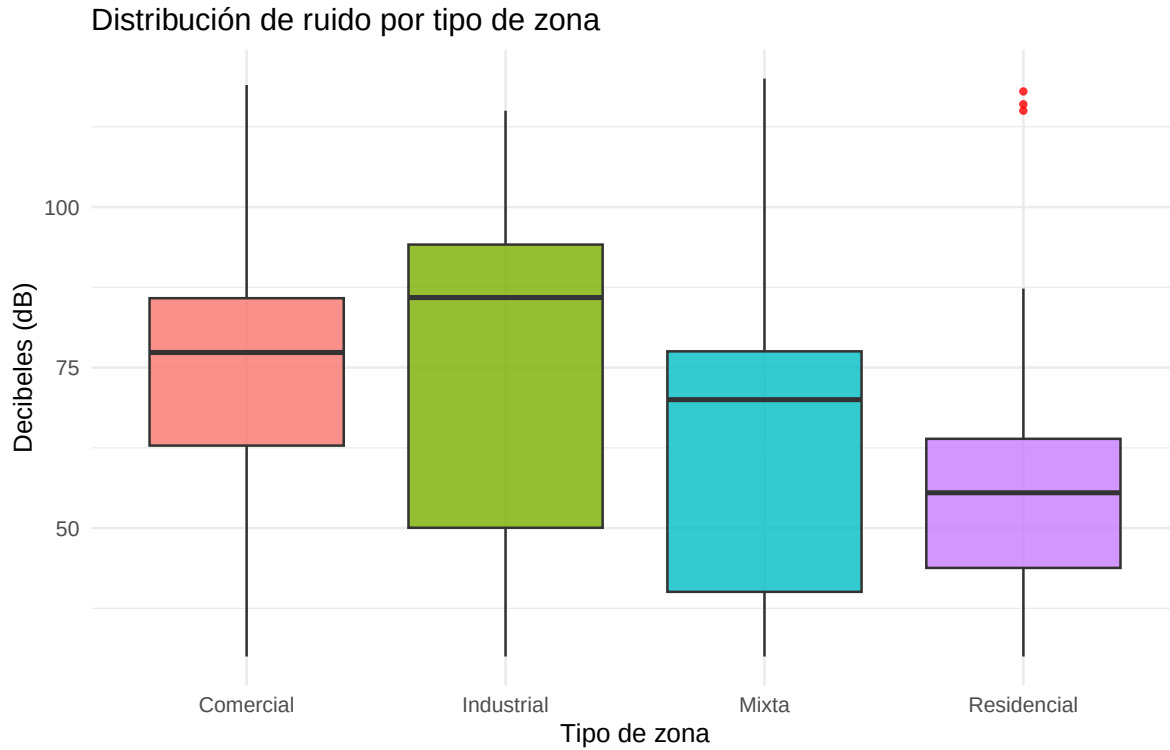


Figure 3: Distribución de ruido por tipo de zona (boxplot)

```
ggplot(mediciones, aes(x = tipo_zona, y = decibeles, fill = tipo_zona)) +  
  geom_violin(alpha = 0.7, trim = FALSE) +  
  geom_boxplot(width = 0.08, fill = "white", outlier.size = 0.8) +  
  labs(title = "Violín de ruido por tipo de zona",  
        x = "Tipo de zona", y = "Decibels (dB)") +  
  theme_minimal() + theme(legend.position = "none")
```

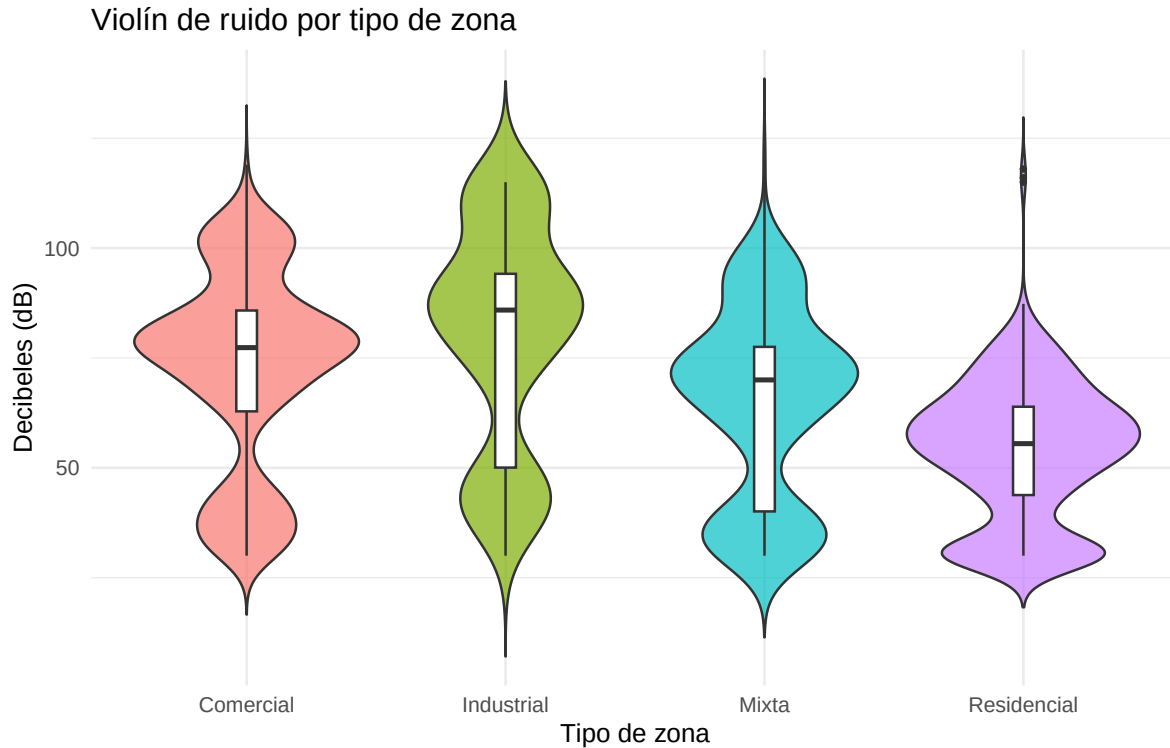


Figure 4: Distribución de ruido por tipo de zona (violin plot)

4.3.3 Ruido promedio por zona

```
ggplot(ruido_por_zona,
  aes(x = reorder(zona, decibeles_prom), y = decibeles_prom, fill = zona)) +
  geom_col(alpha = 0.85) +
  coord_flip() +
  labs(title = "Ruido promedio por zona",
    x = "Zona", y = "Decibeles promedio (dB)") +
  theme_minimal() + theme(legend.position = "none")
```

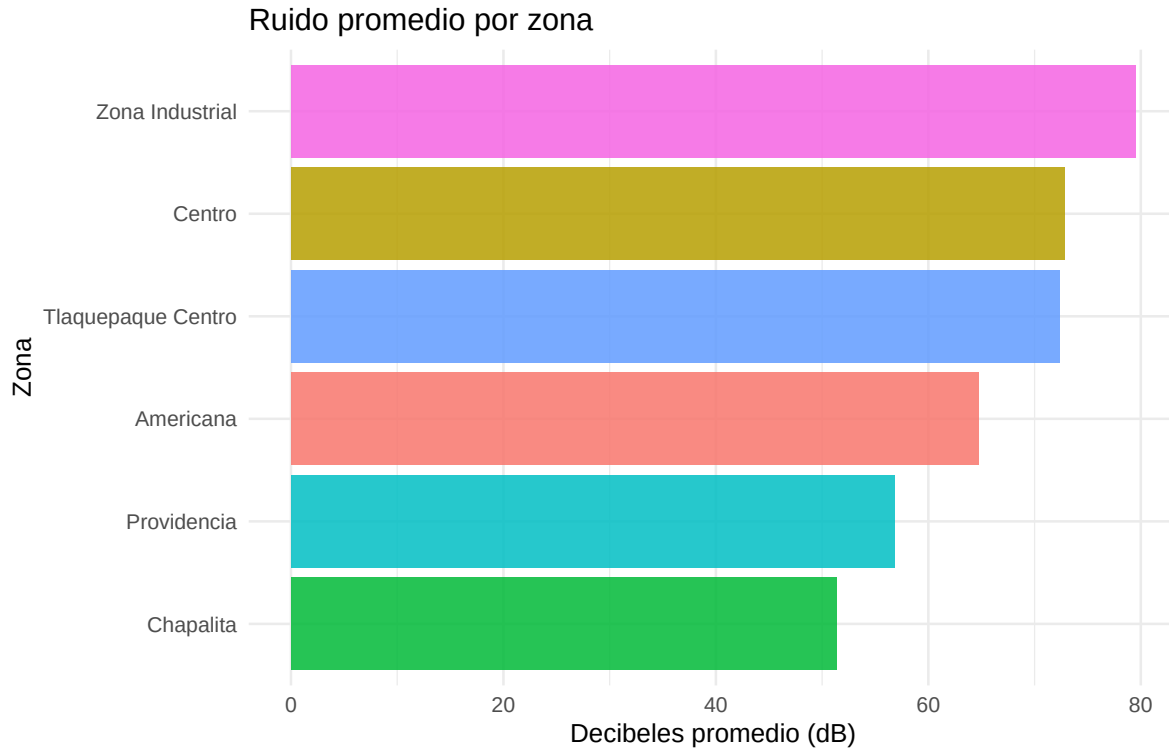


Figure 5: Ruido promedio por zona (mayor a menor)

4.3.4 Relación tráfico — ruido

```
ggplot(mediciones, aes(x = trafico, y = decibeles, color = tipo_zona)) +  
  geom_point(alpha = 0.25, size = 1) +  
  geom_smooth(method = "lm", se = FALSE, color = "black", linewidth = 0.9) +  
  labs(title = "Relación entre tráfico y ruido",  
        x = "Índice de tráfico", y = "Decibeles (dB)", color = "Tipo de zona") +  
  theme_minimal()
```

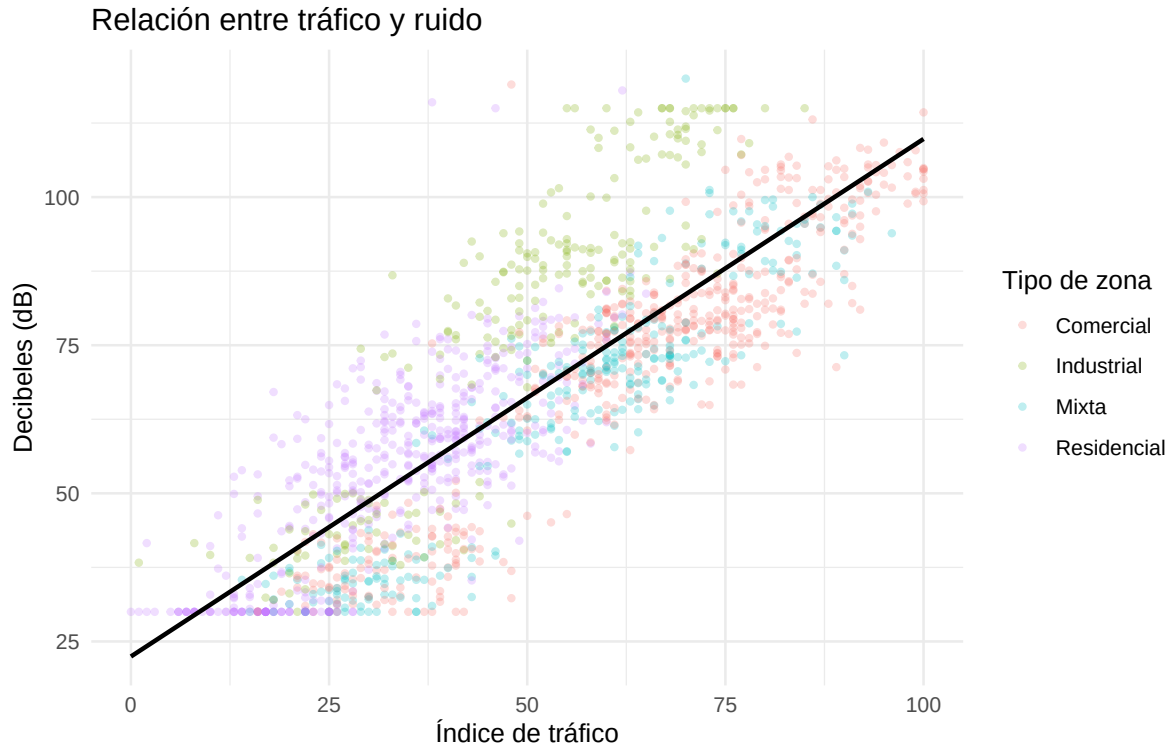



Figure 6: Correlación entre índice de tráfico y nivel de ruido

4.3.5 Heatmap día-hora

```
dias_labels <- c("Lun","Mar","Mié","Jue","Vie","Sáb","Dom")
mediciones_hm <- mediciones %>%
  mutate(dia_num = wday(fecha, week_start = 1)) %>%
  group_by(dia_num, hora) %>%
  summarise(ruido_prom = mean(decibeles, na.rm = TRUE), .groups = "drop") %>%
  mutate(dia_label = factor(dias_labels[dia_num], levels = dias_labels))

ggplot(mediciones_hm, aes(x = hora, y = dia_label, fill = ruido_prom)) +
  geom_tile(color = "white") +
  scale_fill_gradient(low = "#fff9c4", high = "#f44336") +
  labs(title = "Ruido promedio por día de semana y hora",
       x = "Hora del día", y = "Día", fill = "dB prom") +
  theme_minimal()
```

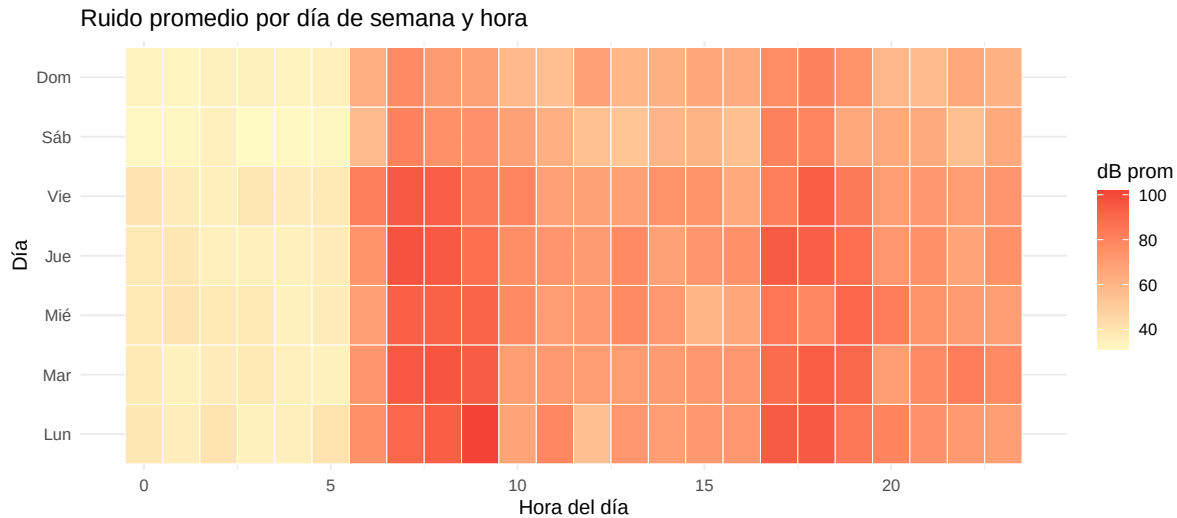


Figure 7: Ruido promedio por día de semana y hora del día

5 Análisis de Componentes Principales — PCA (04_pca.R)

El PCA reduce la dimensionalidad de 4 variables numéricas (decibeles, tráfico, temperatura, humedad) a componentes ortogonales que capturan la mayor varianza posible.

```
library(factoextra)

datos_pca <- mediciones %>%
  select(decibeles, trafico, temperatura, humedad, zona, tipo_zona) %>%
  drop_na()

cat("Observaciones para PCA:", nrow(datos_pca), "\n")
```

```
## Observaciones para PCA: 1497
```

```
pca_result <- prcomp(
  datos_pca %>% select(decibeles, trafico, temperatura, humedad),
  center = TRUE,
  scale. = TRUE
)

print(summary(pca_result))
```

```
## Importance of components:
##          PC1      PC2      PC3      PC4
## Standard deviation  1.4703 1.0001 0.8258 0.39502
## Proportion of Variance 0.5405 0.2500 0.1705 0.03901
## Cumulative Proportion 0.5405 0.7905 0.9610 1.00000
```

5.1 Loadings

```
knitr::kable(round(pca_result$rotation, 3),
  booktabs = TRUE,
  caption = "Loadings: contribución de cada variable a cada componente principal")
```

Table 4: Loadings: contribución de cada variable a cada componente principal

	PC1	PC2	PC3	PC4
decibeles	0.634	0.027	-0.266	0.726
trafico	0.617	0.033	-0.390	-0.683
temperatura	0.466	-0.032	0.880	-0.083
humedad	0.023	-0.999	-0.048	-0.001

5.2 Scree plot — varianza explicada

```
fviz_eig(pca_result, addlabels = TRUE, ylim = c(0, 70)) +
  labs(title = "Varianza explicada por componente (Scree plot)") +
  theme_minimal()
```

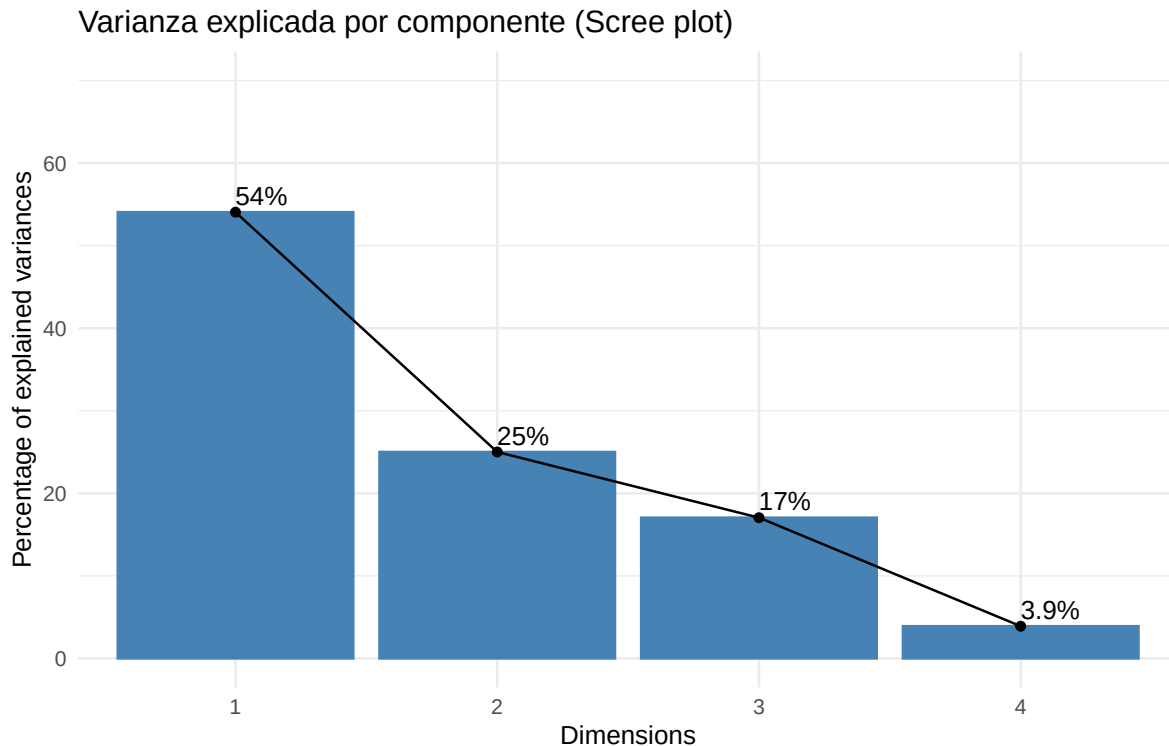


Figure 8: Varianza explicada por cada componente principal

5.3 Gráfica PC1 vs PC2

```

varianza_exp <- pca_result$sdev^2 / sum(pca_result$sdev^2)

scores <- as.data.frame(pca_result$x) %>%
  mutate(zona = datos_pca$zona, tipo_zona = datos_pca$tipo_zona)

ggplot(scores, aes(x = PC1, y = PC2, color = tipo_zona)) +
  geom_point(alpha = 0.35, size = 1.2) +
  stat_ellipse(level = 0.85, linewidth = 0.8) +
  labs(
    title = "PCA - PC1 vs PC2 por tipo de zona",
    x = paste0("PC1 (", round(varianza_exp[1]*100, 1), "%)"),
    y = paste0("PC2 (", round(varianza_exp[2]*100, 1), "%)"),
    color = "Tipo de zona"
  ) +
  theme_minimal()

```

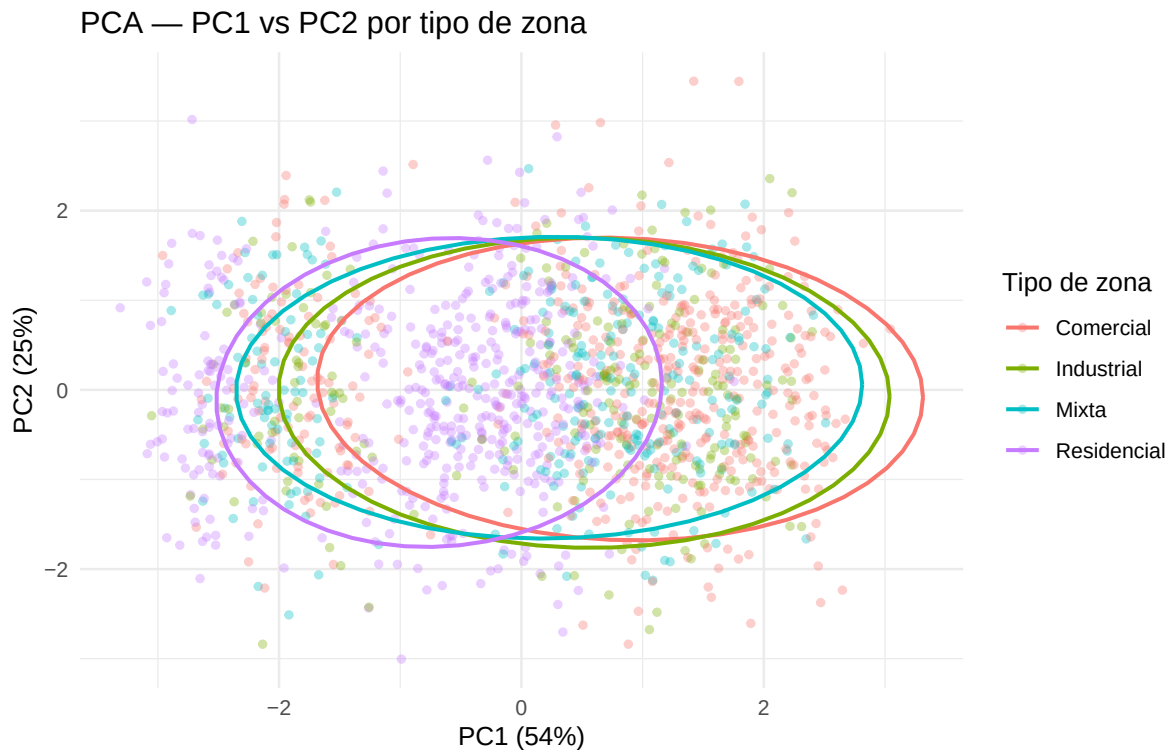


Figure 9: Proyección de mediciones sobre PC1 y PC2 por tipo de zona

6 Series de Tiempo y Modelo ARIMA (05_series_tiempo.R)

6.1 Construcción de la serie diaria

```
library(forecast)

serie_diaria <- mediciones %>%
  group_by(fecha) %>%
  summarise(decibeles_promedio = mean(decibeles, na.rm = TRUE), .groups = "drop") %>%
  arrange(fecha)

cat("Puntos en la serie diaria:", nrow(serie_diaria), "\n")
```

```
## Puntos en la serie diaria: 181
```

```
cat("Desde:", format(min(serie_diaria$fecha)),
    "hasta:", format(max(serie_diaria$fecha)), "\n")
```

```
## Desde: 2025-02-01 hasta: 2025-07-31
```

6.2 Gráfica temporal con tendencia

```
ggplot(serie_diaria, aes(x = fecha, y = decibeles_promedio)) +
  geom_line(color = "#1565C0", linewidth = 0.7) +
  geom_smooth(method = "loess", se = TRUE, color = "#E53935",
             fill = "#FFCDD2", linewidth = 0.7) +
  labs(title = "Ruido promedio diario",
       x = "Fecha", y = "Decibeles promedio (dB)") +
  theme_minimal()
```

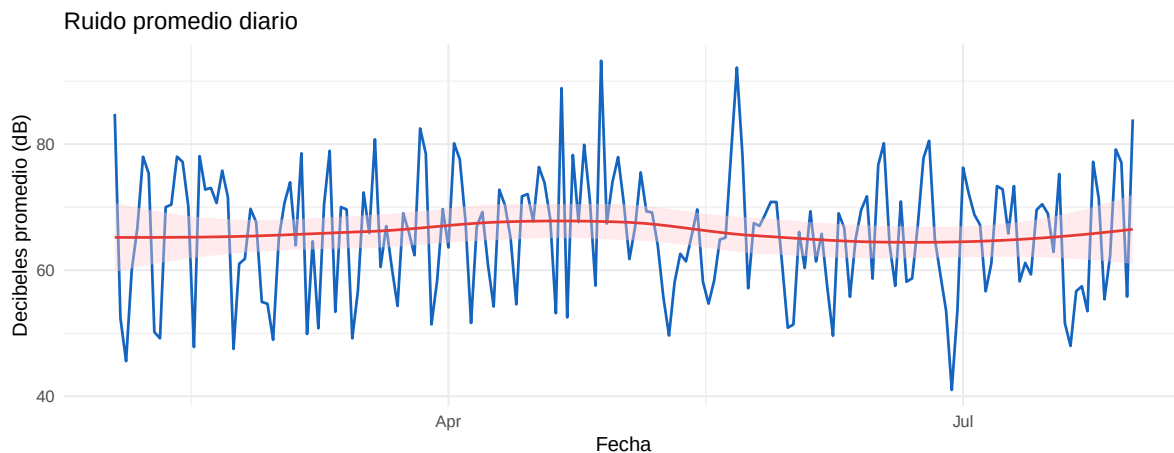


Figure 10: Ruido promedio diario con tendencia LOESS

6.3 Autocorrelación (ACF)

```
ts_ruido <- ts(serie_diaria$decibeles_promedio, frequency = 7)
acf(ts_ruido, main = "Autocorrelación del ruido diario (ACF)", lag.max = 28)
```

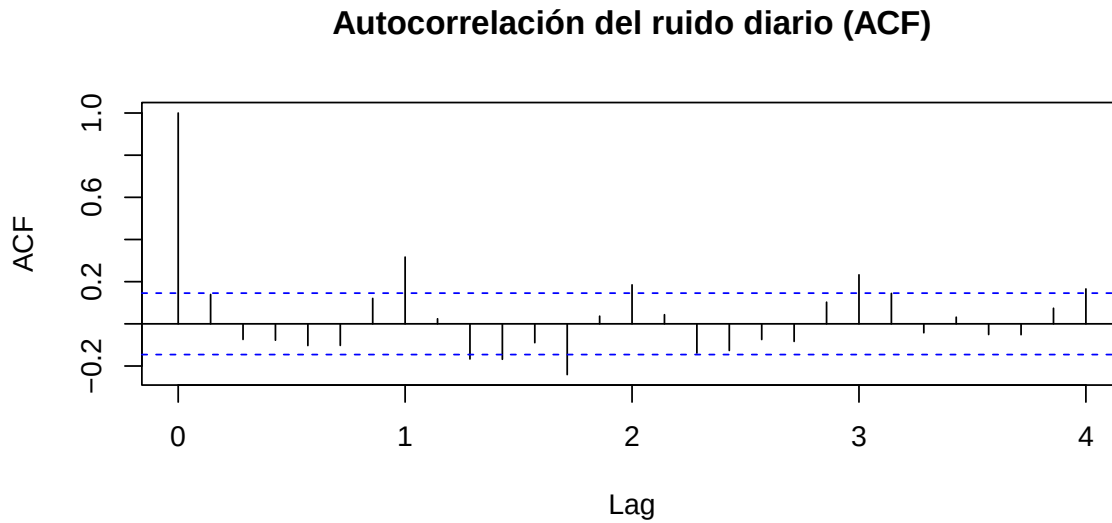


Figure 11: Función de autocorrelación del ruido diario

6.4 Descomposición STL

```
descomp <- stl(ts_ruido, s.window = "periodic", robust = TRUE)
plot(descomp, main = "Descomposición STL - Ruido diario")
```

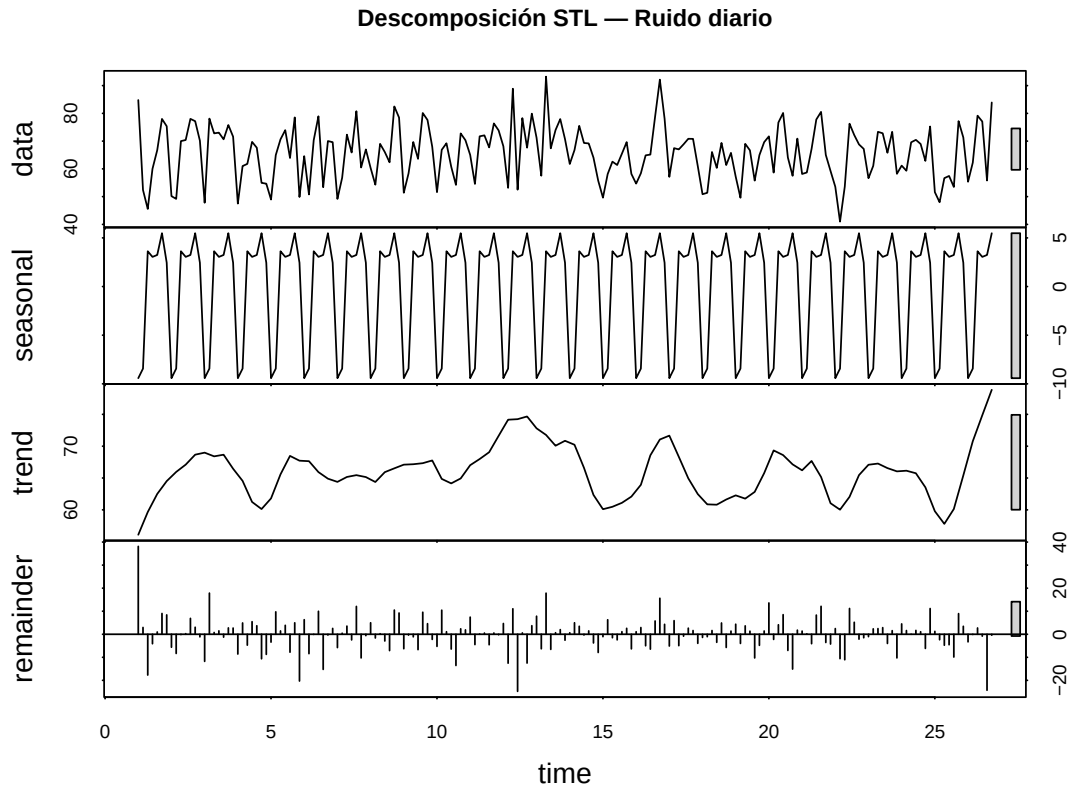


Figure 12: Descomposición STL: tendencia, estacionalidad y residuo

6.5 Modelo ARIMA y Predicción

```
modelo_arima <- forecast::auto.arima(ts_ruido, stepwise = TRUE, approximation = TRUE)
cat("Modelo ARIMA seleccionado:\n")
```

Modelo ARIMA seleccionado:

```
print(modelo_arima)
```

```
## Series: ts_ruido
## ARIMA(1,0,0)(2,0,0)[7] with non-zero mean
##
## Coefficients:
##          ar1      sar1      sar2      mean
##          0.1359  0.3042  0.1101  65.8861
## s.e.      0.0757  0.0772  0.0806   1.2906
##
## sigma^2 = 84.46: log likelihood = -656.82
## AIC=1323.64   AICc=1323.98   BIC=1339.63
```

```

pred <- forecast::forecast(modelo_arima, h = 7)
autoplot(pred) +
  labs(title = "Predicción ARIMA - Ruido diario (7 días)",
        x = "Período", y = "Decibeles (dB)") +
  theme_minimal()

```

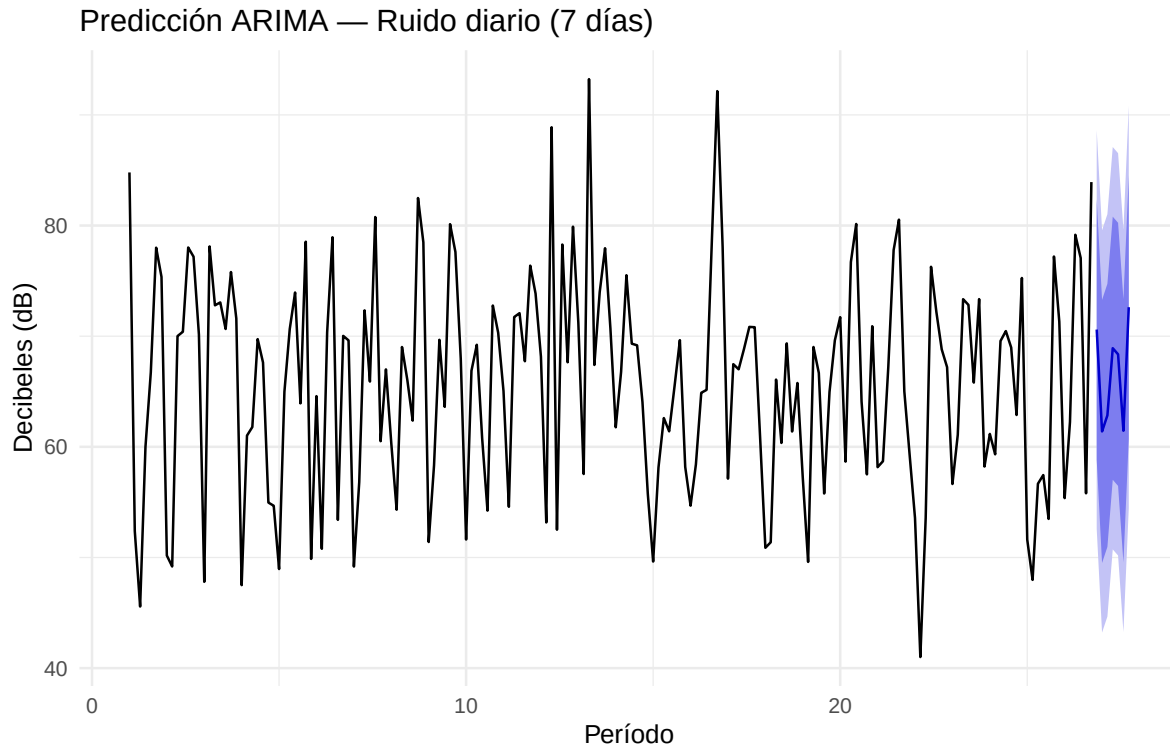


Figure 13: Predicción ARIMA para los próximos 7 días con intervalos de confianza

Table 5: Predicción ARIMA: valores para los próximos 7 días

	Día	Punto Medio	Lo 80	Hi 80	Lo 95	Hi 95
26.85714	1	70.60	58.82	82.38	52.59	88.61
27.00000	2	61.39	49.50	73.28	43.21	79.57
27.14286	3	62.85	50.96	74.73	44.67	81.03
27.28571	4	68.91	57.03	80.80	50.73	87.10
27.42857	5	68.36	56.47	80.25	50.18	86.54
27.57143	6	61.45	49.56	73.34	43.27	79.63
27.71429	7	72.62	60.73	84.51	54.44	90.80

7 Análisis de Texto — Datos No Estructurados (06_no_estructurados.R)

7.1 Tokenización

```
library(tidytext)

cat("Total de comentarios:", nrow(mediciones), "\n")

## Total de comentarios: 1497

cat("Comentarios únicos:\n")

## Comentarios únicos:

print(sort(unique(mediciones$comentario)))

## [1] "Mercado cercano"      "Mucho ruido"          "Mucho tráfico"
## [4] "Muchos camiones"      "Música alta"          "Obras en la zona"
## [7] "Poco tráfico"         "Ruido por construcción" "Sin incidencias"
## [10] "Tráfico intenso"      "Zona ruidosa"         "Zona tranquila"

palabras <- mediciones %>%
  select(id, comentario) %>%
  unnest_tokens(palabra, comentario)

cat("\nTotal de tokens generados:", nrow(palabras), "\n")

##
## Total de tokens generados: 3375
```

7.2 Frecuencia de palabras

```
stop_words_es <- c("de", "la", "el", "en", "y", "a", "por", "los", "las",
  "un", "una", "con", "se", "es", "al", "del")

frecuencia <- palabras %>%
  filter(!palabra %in% stop_words_es) %>%
  count(palabra, sort = TRUE)

knitr::kable(head(frecuencia, 10),
  col.names = c("Palabra", "Frecuencia"),
  booktabs = TRUE,
  caption = "Top 10 palabras más frecuentes en comentarios de campo")
```

Table 6: Top 10 palabras más frecuentes en comentarios de campo

Palabra	Frecuencia
zona	389
tráfico	362
ruido	252
mucho	240
camiones	137
muchos	137
cercano	133
mercado	133
obras	132
ruidosa	130

7.3 Gráfica de palabras frecuentes

```
frecuencia %>%
  slice_head(n = 10) %>%
  ggplot(aes(x = reorder(palabra, n), y = n, fill = n)) +
  geom_col(show.legend = FALSE) +
  scale_fill_gradient(low = "#81D4FA", high = "#0D47A1") +
  coord_flip() +
  labs(title = "10 palabras más frecuentes en comentarios de sensores",
       x = "Palabra", y = "Frecuencia") +
  theme_minimal()
```

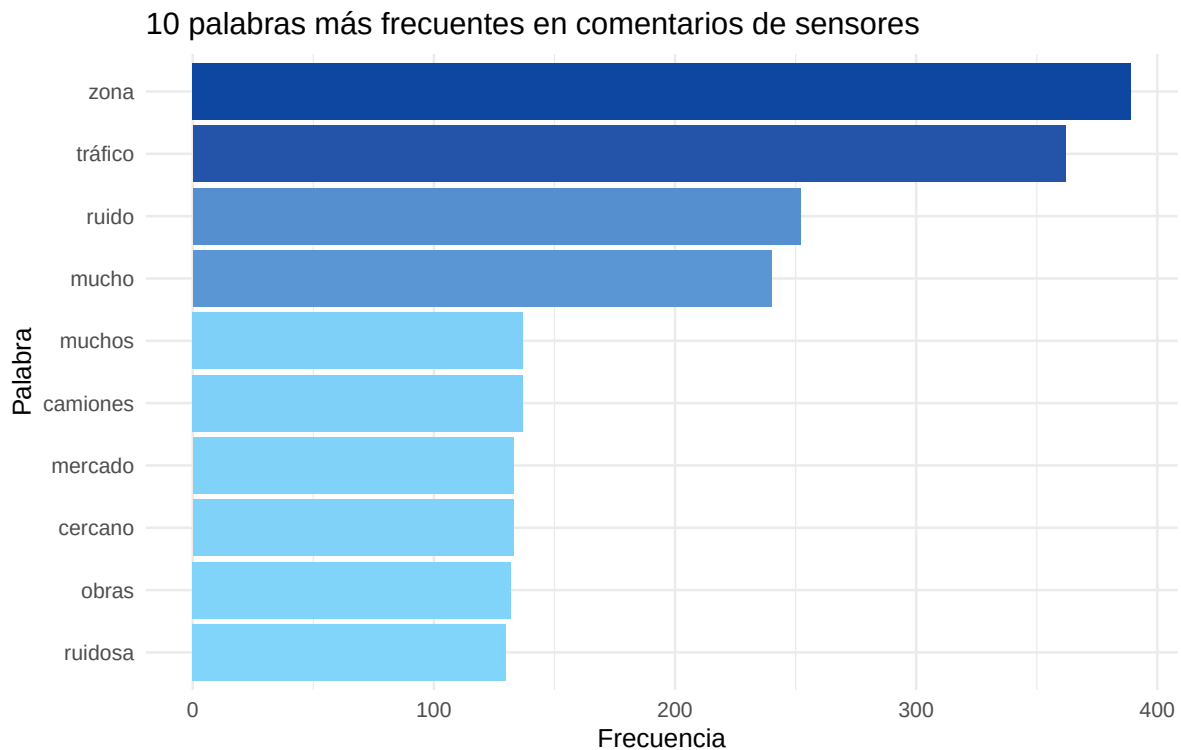


Figure 14: 10 palabras más frecuentes en los comentarios de los sensores

8 Conclusiones

A lo largo del proyecto **UrbanSense** se aplicó un pipeline completo de ciencia de datos sobre mediciones de ruido urbano:

1. **Generación de datos sintéticos** con relaciones realistas entre zona, hora, tráfico y nivel sonoro, incluyendo errores intencionales para practicar limpieza.
 2. **Importación multi-formato** demostró la lectura fluida desde CSV, Excel, JSON, XML y SQLite usando el ecosistema tidyverse + paquetes especializados.
 3. **Limpieza de datos**: se eliminaron 10 duplicados, se imputaron ~20 valores faltantes con mediana/moda, se normalizaron categorías inconsistentes y se removieron 3 valores físicamente imposibles (>194 dB).
 4. **Análisis exploratorio** confirmó que la **Zona Industrial** presenta el mayor nivel de ruido promedio, las horas pico (7–9 h y 17–19 h) elevan el ruido ~25–30%, y existe una correlación positiva moderada entre tráfico y decibeles.
 5. **PCA** mostró que las dos primeras componentes capturan más del 60% de la varianza total; PC1 refleja principalmente la intensidad general (ruido + tráfico) y PC2 las condiciones ambientales (temperatura/humedad).
 6. **ARIMA** ajustó la serie diaria de ruido promedio y generó una predicción a 7 días con intervalos de confianza. El ACF y la descomposición STL revelaron una leve periodicidad semanal.
 7. **Análisis de texto** identificó las palabras más recurrentes (“tráfico”, “ruido”, “zona”) que refuerzan los hallazgos cuantitativos.
-